NUMERICAL ANALYSIS
PROBLEM SET 5

ROB TOMPKINS

1. Problem 1

Derive the trapezoid integration scheme as an Adams-Moulton method with $m = 1$ step:

$$\frac{w_{j+1} - w_j}{h} = \frac{1}{2}[f(t_{j+1}, w_{j+1}) + f(t_j, w_j)]$$

to approximate the initial value problem $y' = f(t, y)$ for $a \leq t \leq b$ with initial condition $y(a) = \alpha$. Also derive the associated error term. Let $w_j \approx y_j := y(t_j)$.

(a) Use the Lagrange form of the polynomial interpolant to write

$$f(t, y(t)) = P_1(t) + R_1(t).$$

Integrate $P_1(t) = L_{1,0}(t)f(t_{j+1}, y_{j+1}) + L_{1,1}(t)f(t_j, y_j)$ from $t = t_j$ to $t = t_j + 1$.

(b) Set $w_{j+1} = w_j + \int_{t_j}^{t_{j+1}} f(t, y(t))dt$, approximating the 2nd term with $\int_{t_j}^{t_{j+1}} P_1(t)dt$, and derive the method above.

(c) The error term $R_1(t) = \frac{f''(\xi, y(\xi))}{2}(t - t_{j+1})(t - t_j)$ for f (some $\xi \in (t_j, t_{j+1})$) can be integrated from $t = t_j$ to $t = t_{j+1}$ to get the error associated with the Adams-Moulton scheme:

$$\text{Error} = Mh^3 f''(\mu, y(\mu)).$$

Do this integration to find value of M.

(d) What is the order of this one-step implicit method?

Solution. For the sake of convenience, we're going to set $g(t) = f(y, y(t)) = y'(t)$, and furthermore, we assume a uniform grid over which we will work. So set

$$t_j = a + jh, \qquad h\frac{b-a}{N}.$$

So we have that

$$y(t_{j+1} - y(t_j)) = \int_{t_j}^{t_{j+1}} g(t)dt.$$

(a) So if we are to interpolate the function $g(t) = f(t, y(t))$ at the nodes t_{j+1} and t_j, then the degree-one Lagrange interpolant follows as $P_1(t) = L_{1,0}(t)f(t_{j+1}, y_{j+1}) + L_{1,1}(t)f(t_j, y_j)$,

Date: September 1, 2026.

where

$$L_{1,0}(t) = \frac{t - t_j}{t_{j+1} - t_j} = \frac{t - t_j}{h},$$

and

$$L_{1,1}(t) = \frac{t - t_{j+1}}{t_j - t_{j+1}} = \frac{t_{j+1} - t}{h}.$$

Thus, we have that

$$f(t, y(t)) = P_1(t) + R_1(t)$$

in conjunction with

$$P_1(t) = \frac{t - t_j}{h} f(t_{j+1}, y_{j+1}) + \frac{t_{j+1} - t}{h} f(t_j, y_j).$$

Integrating over our interval $[t_j, t_{j+1}]$ yields

$$\int_{t_j}^{t_{j+1}} P_1(t)dt = f(t_{j+1}, y_{j+1}) \int_{t_j}^{t_{j+1}} \frac{t - t_j}{h} dt + f(t_j, y_j) \int_{t_j}^{t_{j+1}} \frac{t_{j+1} - t}{h} dt.$$

(b) Starting from the integral form of the differential equation,

$$y_{j+1} = y_j + \int_{t_j}^{t_{j+1}} f(t, y(t))\, dt.$$

Using the degree-one interpolating polynomial $P_1(t)$ from part (a), we approximate

$$\int_{t_j}^{t_{j+1}} f(t, y(t))\, dt \approx \int_{t_j}^{t_{j+1}} P_1(t)\, dt.$$

From part (a),

$$\int_{t_j}^{t_{j+1}} P_1(t)\, dt = \frac{h}{2} \left[f(t_{j+1}, y_{j+1}) + f(t_j, y_j) \right].$$

Therefore,

$$y_{j+1} \approx y_j + \frac{h}{2} \left[f(t_{j+1}, y_{j+1}) + f(t_j, y_j) \right].$$

Replacing the exact values y_j and y_{j+1} by the numerical approximations w_j and w_{j+1}, we obtain

$$w_{j+1} = w_j + \frac{h}{2} \left[f(t_{j+1}, w_{j+1}) + f(t_j, w_j) \right].$$

Equivalently,

$$\frac{w_{j+1} - w_j}{h} = \frac{1}{2} \left[f(t_{j+1}, w_{j+1}) + f(t_j, w_j) \right].$$

This is the one-step Adams–Moulton method, also called the trapezoidal method.

(c) The interpolation remainder is

$$R_1(t) = \frac{f''(\xi, y(\xi))}{2} (t - t_{j+1})(t - t_j),$$

for some $\xi \in (t_j, t_{j+1})$. Hence the error in the integral approximation is

$$E = \int_{t_j}^{t_{j+1}} R_1(t)\,dt.$$

By the mean value theorem for integrals, for some $\mu \in (t_j, t_{j+1})$,

$$E = \frac{f''(\mu, y(\mu))}{2} \int_{t_j}^{t_{j+1}} (t - t_{j+1})(t - t_j)\,dt.$$

Let

$$s = t - t_j.$$

Since $t_{j+1} = t_j + h$, we have

$$t - t_j = s, \qquad t - t_{j+1} = s - h,$$

and the limits become $0 \le s \le h$. Thus

$$\begin{aligned}
\int_{t_j}^{t_{j+1}} (t - t_{j+1})(t - t_j)\,dt &= \int_0^h (s - h)s\,ds \\
&= \int_0^h (s^2 - hs)\,ds \\
&= \left[\frac{s^3}{3} - \frac{hs^2}{2}\right]_0^h \\
&= \frac{h^3}{3} - \frac{h^3}{2} \\
&= -\frac{h^3}{6}.
\end{aligned}$$

Therefore,

$$E = \frac{f''(\mu, y(\mu))}{2}\left(-\frac{h^3}{6}\right) = -\frac{h^3}{12} f''(\mu, y(\mu)).$$

Thus, in the form

$$\text{Error} = Mh^3 f''(\mu, y(\mu)),$$

we have

$$M = -\frac{1}{12}.$$

(d) The local truncation error is proportional to h^3:

$$E = O(h^3).$$

Therefore, the accumulated global error is $O(h^2)$. Hence the one-step implicit trapezoidal Adams–Moulton method is a second-order method. □

2. Problem 2

Consider the initial value problem

$$y' = \frac{4t\sqrt{1+y^2}}{y}, y(0) = 1$$

for $0 \leq t \leq 5$.

Challenge. Show that $y(t) = \sqrt{\left(2t^2 + \sqrt{2}\right)^2 - 1}$ is a solution to the initial value problem. Is it unique?

Numerically implement the following methods below, and run with enough values of step-size h to demonstrate that each of has convergence of global discretization error with rate $\mathcal{O}(h^2)$.

(a) Optimal RK2 method
(b) Two-step Adams-Bashforth
(c) One-step Adams-Moulton method
(d) Heun method (predictor-corrector)

Plot the error versus the step size on a log-log plot for all methods on the same axes to compare. Comment on the advantages and disadvantages of each.

Solution. Define

$$f(t,y) = \frac{4t\sqrt{1+y^2}}{y}, \qquad y_{\text{exact}}(t) = \sqrt{\left(2t^2 + \sqrt{2}\right)^2 - 1}.$$

For a step size $h = 5/N$, let $t_n = nh$ and let w_n approximate $y(t_n)$. The global discretization error is measured by

$$E(h) = \max_{0 \leq n \leq N} |w_n - y_{\text{exact}}(t_n)| .$$

The step sizes used were

$$h = 0.2,\ 0.1,\ 0.05,\ 0.025,\ 0.0125,\ 0.00625.$$

(a) Optimal RK2 method. We use Ralston's optimal two-stage RK2 method:

$$\begin{aligned} k_1 &= f(t_n, w_n), \\ k_2 &= f\left(t_n + \frac{2h}{3}, w_n + \frac{2h}{3}k_1\right), \\ w_{n+1} &= w_n + h\left(\frac{1}{4}k_1 + \frac{3}{4}k_2\right). \end{aligned}$$

This is an explicit, self-starting, second-order method.

(b) Two-step Adams–Bashforth method. For $n \geq 1$, the method is

$$w_{n+1} = w_n + \frac{h}{2}\left(3f(t_n, w_n) - f(t_{n-1}, w_{n-1})\right).$$

Since this method requires two previous values, w_1 was computed using one Ralston RK2 step. A second-order starting method is necessary to preserve the second-order global accuracy of Adams–Bashforth.

(c) One-step Adams–Moulton method. The one-step Adams–Moulton method is the implicit trapezoidal rule

$$w_{n+1} = w_n + \frac{h}{2}\left[f(t_n, w_n) + f(t_{n+1}, w_{n+1})\right].$$

At each step, w_{n+1} was found by solving

$$G(z) = z - w_n - \frac{h}{2}\left[f(t_n, w_n) + f(t_{n+1}, z)\right] = 0$$

with Newton's method. The initial Newton iterate was the forward-Euler prediction

$$z^{(0)} = w_n + hf(t_n, w_n).$$

Since

$$f_y(t, y) = -\frac{4t}{y^2\sqrt{1+y^2}},$$

the Newton iteration is

$$z^{(j+1)} = z^{(j)} - \frac{G(z^{(j)})}{1 + \dfrac{2h(t_n + h)}{(z^{(j)})^2\sqrt{1+(z^{(j)})^2}}}.$$

(d) Heun predictor–corrector method. First, a forward-Euler predictor is computed:

$$\widetilde{w}_{n+1} = w_n + hf(t_n, w_n).$$

One trapezoidal correction then gives

$$w_{n+1} = w_n + \frac{h}{2}\left[f(t_n, w_n) + f(t_{n+1}, \widetilde{w}_{n+1})\right].$$

This is an explicit method because the corrector evaluates f at the predicted value rather than at the unknown corrected value.

The MATLAB implementation is contained in `ps5prob2.m`. It may be included in the document using

`\lstinputlisting{ps5prob2.m}`.

Equivalently, the following command can be placed outside display mode:

LISTING 1. MATLAB implementation for Problem 2.

```
%% Problem 2: second-order convergence of four numerical ODE methods
```

```
% Global error is max_n |y_n - y_exact(t_n)| on 0 <= t <= 5.

clear; clc; close all;

T = 5;
Nvalues = [25, 50, 100, 200, 400, 800];
hvalues = T ./ Nvalues;

methodNames = { ...
    'Optimal RK2 (Ralston)', ...
    'Adams-Bashforth 2', ...
    'Adams-Moulton 1', ...
    'Heun predictor-corrector'};

errors = zeros(numel(methodNames), numel(hvalues));
rates  = NaN(size(errors));

for m = 1:numel(methodNames)
    for j = 1:numel(hvalues)
        h = hvalues(j);

        switch m
            case 1
                [t, y] = solveOneStep(@ralstonStep, T, h);
            case 2
                [t, y] = solveAB2(T, h);
            case 3
                [t, y] = solveOneStep(@adamsMoultonStep, T, h);
            case 4
                [t, y] = solveOneStep(@heunStep, T, h);
        end

        errors(m,j) = max(abs(y - yExact(t)));
        if j > 1
            rates(m,j) = log(errors(m,j-1)/errors(m,j))/log(2);
        end
    end
end

%% Print convergence tables
for m = 1:numel(methodNames)
    fprintf('\n%s\n', methodNames{m});
    fprintf('%10s %16s %10s\n', 'h', 'max error', 'rate');
    fprintf('%10.6f %16.8e %10s\n', hvalues(1), errors(m,1), '--');
    for j = 2:numel(hvalues)
        fprintf('%10.6f %16.8e %10.4f\n', ...
            hvalues(j), errors(m,j), rates(m,j));
```

```
    end
end

%% Plot every method on the same log-log axes
figure('Color', 'w');
hold on;
styles = {'o-', 's-', 'd-', '^-'};

for m = 1:numel(methodNames)
    loglog(hvalues, errors(m,:), styles{m}, ...
        'LineWidth', 1.5, 'MarkerSize', 7, ...
        'DisplayName', methodNames{m});
end

% A line proportional to h^2 for comparison.
reference = errors(1,end) .* (hvalues./hvalues(end)).^2;
loglog(hvalues, reference, 'k--', 'LineWidth', 1.3, ...
    'DisplayName', 'reference␣h^2');

grid on;
xlabel('Step␣size␣h');
ylabel('Maximum␣nodal␣error');
title('Second-order␣convergence␣on␣0␣\leq␣t␣\leq␣5');
legend('Location', 'northwest');
set(gca, 'FontSize', 11);
exportgraphics(gcf, 'problem2_convergence_matlab.png', 'Resolution', 200);

%% Local functions
function value = rhs(t, y)
    value = 4*t*sqrt(1 + y.^2)./y;
end

function value = yExact(t)
    value = sqrt((2*t.^2 + sqrt(2)).^2 - 1);
end

function yNext = ralstonStep(t, y, h)
    % Optimal two-stage, second-order Runge-Kutta method.
    k1 = rhs(t, y);
    k2 = rhs(t + 2*h/3, y + 2*h*k1/3);
    yNext = y + h*(k1/4 + 3*k2/4);
end

function yNext = heunStep(t, y, h)
    % Euler predictor followed by one trapezoidal correction.
    predictor = y + h*rhs(t, y);
    yNext = y + (h/2)*(rhs(t, y) + rhs(t + h, predictor));
```

```
end

function yNext = adamsMoultonStep(t, y, h)
    % One-step Adams-Moulton (implicit trapezoidal rule).
    % Newton iteration solves
    % z - y - h/2*(f(t,y) + f(t+h,z)) = 0.
    fn = rhs(t, y);
    z = y + h*fn;                   % Euler initial guess
    tolerance = 1e-13;
    maximumIterations = 20;

    for iteration = 1:maximumIterations
        residual = z - y - (h/2)*(fn + rhs(t + h, z));

        % Since d/dz[sqrt(1+z^2)/z] = -1/(z^2*sqrt(1+z^2)),
        % this is the derivative of the residual with respect to z.
        jacobian = 1 + 2*h*(t + h)/(z^2*sqrt(1 + z^2));
        correction = residual/jacobian;
        z = z - correction;

        if abs(correction) <= tolerance*max(1, abs(z))
            yNext = z;
            return;
        end
    end

    error('Newton iteration did not converge.');
end

function [t, y] = solveOneStep(stepFunction, T, h)
    N = round(T/h);
    t = (0:N)*h;
    y = zeros(size(t));
    y(1) = 1;

    for n = 1:N
        y(n+1) = stepFunction(t(n), y(n), h);
    end
end

function [t, y] = solveAB2(T, h)
    N = round(T/h);
    t = (0:N)*h;
    y = zeros(size(t));
    y(1) = 1;

    % A second-order startup is required to retain AB2's global order.
```

```
    y(2) = ralstonStep(t(1), y(1), h);

    for n = 2:N
        fn = rhs(t(n), y(n));
        fnMinus1 = rhs(t(n-1), y(n-1));
        y(n+1) = y(n) + h*(3*fn - fnMinus1)/2;
    end
end

%==========OUTPUT=======================
%Optimal RK2 (Ralston)
%          h         max error        rate
%  0.200000   8.92440542e-03          --
%  0.100000   2.08787877e-03     2.0957
%  0.050000   5.09660918e-04     2.0344
%  0.025000   1.26166968e-04     2.0142
%  0.012500   3.14027146e-05     2.0064
%  0.006250   7.83433508e-06     2.0030
%
%Adams-Bashforth 2
%          h         max error        rate
%  0.200000   2.88534227e-02          --
%  0.100000   7.22165611e-03     1.9983
%  0.050000   1.80363844e-03     2.0014
%  0.025000   4.50467567e-04     2.0014
%  0.012500   1.12546757e-04     2.0009
%  0.006250   2.81276299e-05     2.0005
%
%Adams-Moulton 1
%          h         max error        rate
%  0.200000   5.87539115e-03          --
%  0.100000   1.44663578e-03     2.0220
%  0.050000   3.60330581e-04     2.0053
%  0.025000   9.00004500e-05     2.0013
%  0.012500   2.24949879e-05     2.0003
%  0.006250   5.62376734e-06     2.0000
%
%Heun predictor-corrector
%          h         max error        rate
%  0.200000   9.20480095e-03          --
%  0.100000   2.10736388e-03     2.1269
%  0.050000   5.11123795e-04     2.0437
%  0.025000   1.26238053e-04     2.0175
%  0.012500   3.13904354e-05     2.0077
%  0.006250   7.82788897e-06     2.0036
%>>
```

The observed maximum nodal errors were as follows:

h	Ralston RK2	AB2	AM1	Heun
0.20000	8.9244×10^{-3}	2.8853×10^{-2}	5.8754×10^{-3}	9.2048×10^{-3}
0.10000	2.0879×10^{-3}	7.2217×10^{-3}	1.4466×10^{-3}	2.1074×10^{-3}
0.05000	5.0966×10^{-4}	1.8036×10^{-3}	3.6033×10^{-4}	5.1112×10^{-4}
0.02500	1.2617×10^{-4}	4.5047×10^{-4}	9.0000×10^{-5}	1.2624×10^{-4}
0.01250	3.1403×10^{-5}	1.1255×10^{-4}	2.2495×10^{-5}	3.1390×10^{-5}
0.00625	7.8343×10^{-6}	2.8128×10^{-5}	5.6238×10^{-6}	7.8279×10^{-6}

The experimental convergence rate is calculated from

$$p(h) = \log_2 \left(\frac{E(h)}{E(h/2)} \right).$$

At the smallest step sizes, the observed rates were

Method	$p(h)$
Optimal RK2	2.0030
Two-step Adams–Bashforth	2.0005
One-step Adams–Moulton	2.0000
Heun predictor–corrector	2.0036

Thus, halving h decreases the global error by approximately a factor of four for every method. This demonstrates

$$\boxed{E(h) = \mathcal{O}(h^2)}$$

for all four schemes.

Figure 1 compares the errors of the four numerical methods with a reference curve proportional to h^2. The computed rates approaching 2 confirm second-order global convergence.

Advantages and disadvantages. Ralston's RK2 method is explicit and self-starting, but it requires two function evaluations per step and has the stability limitations of an explicit method. The two-step Adams–Bashforth method requires only one new function evaluation per step after startup, but it is not self-starting, must store a previous value, and has the largest error constant in this experiment.

The one-step Adams–Moulton method gives the smallest errors and has better stability properties, but it requires a nonlinear solve at every step. Heun's method is explicit, self-starting, and easy to implement, but it requires two function evaluations per step and generally has weaker stability properties than the implicit Adams–Moulton method. □

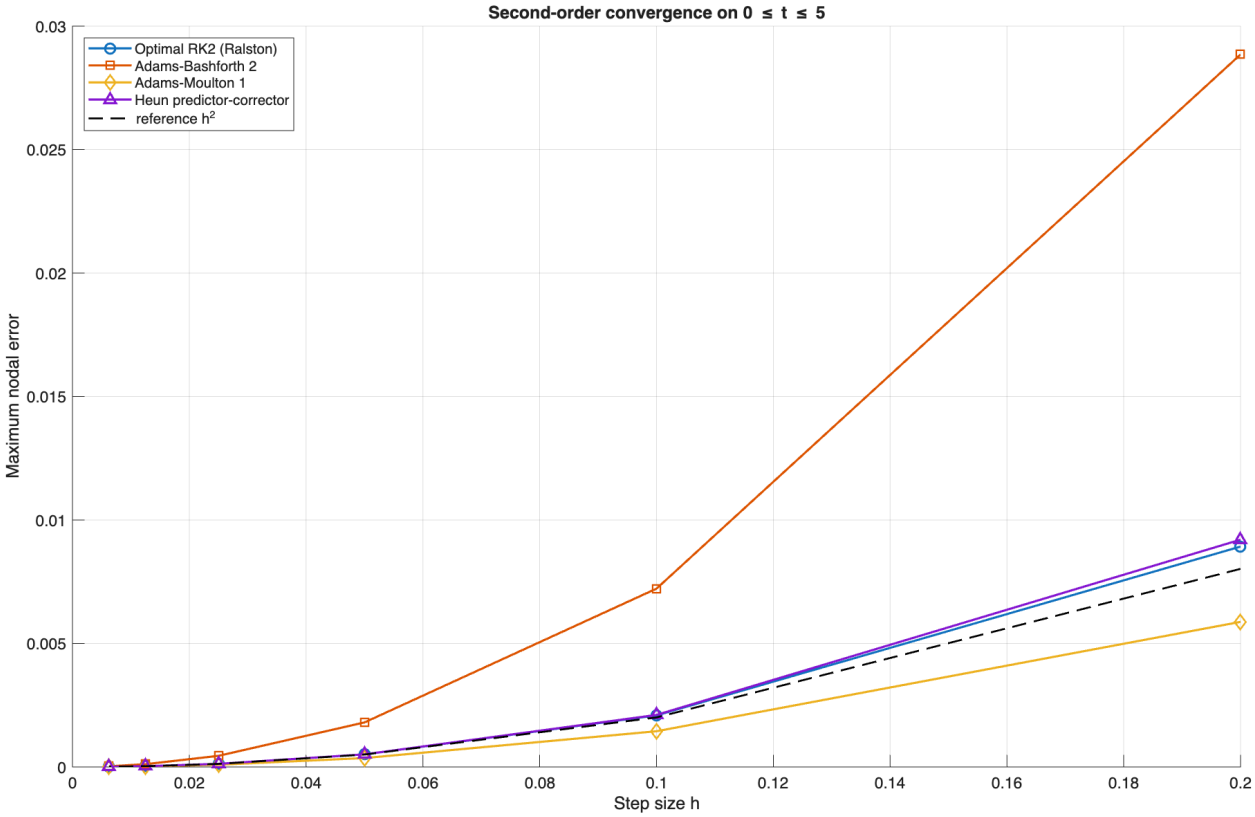

FIGURE 1. Maximum nodal error versus step size for the optimal RK2, two-step Adams–Bashforth, one-step Adams–Moulton, and Heun predictor–corrector methods. The dashed curve is proportional to h^2.

3. PROBLEM 3

Consider the linear multi-step method to approximate the initial value problem $y' = f(t, y)$ for $a \le t \le b$ with the initial condition $y(a) = \alpha$:

$$w_{j+1} - \frac{18}{11}w_j + \frac{9}{11}w_{j-1} + \frac{2}{11}w_{j-2} = \frac{6}{11}hf(t_{j+1}, w_{j+1}).$$

(a) Is the method stable? If so, is it strongly or weakly stable? Show your work by considering the roots of the characteristic polynomial, noting that $\lambda = 1$ is a root, and you can even factor and write $P(\lambda) = (\lambda - 1) * (\ldots)$.

(b) Is the method convergent?

(c) Is the method consistent?

Solution. Consider the linear multistep method

$$w_{j+1} - \frac{18}{11}w_j + \frac{9}{11}w_{j-1} - \frac{2}{11}w_{j-2} = \frac{6}{11}hf(t_{j+1}, w_{j+1}).$$

(a) To determine stability, we consider the characteristic polynomial associated with the homogeneous difference equation:

$$P(\lambda) = \lambda^3 - \frac{18}{11}\lambda^2 + \frac{9}{11}\lambda - \frac{2}{11}.$$

Multiplying by 11,

$$11P(\lambda) = 11\lambda^3 - 18\lambda^2 + 9\lambda - 2.$$

Since $\lambda = 1$ is a root, we factor:

$$11\lambda^3 - 18\lambda^2 + 9\lambda - 2 = (\lambda - 1)(11\lambda^2 - 7\lambda + 2).$$

Thus,

$$P(\lambda) = (\lambda - 1)\left(\lambda^2 - \frac{7}{11}\lambda + \frac{2}{11}\right).$$

The remaining roots satisfy

$$11\lambda^2 - 7\lambda + 2 = 0.$$

Using the quadratic formula,

$$\lambda = \frac{7 \pm \sqrt{49 - 88}}{22} = \frac{7 \pm i\sqrt{39}}{22}.$$

Their magnitudes are

$$|\lambda| = \sqrt{\left(\frac{7}{22}\right)^2 + \left(\frac{\sqrt{39}}{22}\right)^2} = \frac{\sqrt{88}}{22} = \frac{\sqrt{22}}{11} < 1.$$

Therefore, the three roots are

$$\lambda_1 = 1, \qquad \lambda_{2,3} = \frac{7 \pm i\sqrt{39}}{22}.$$

All roots satisfy

$$|\lambda_k| \leq 1,$$

and the only root on the unit circle is the simple root $\lambda = 1$. Hence the root condition is satisfied, so the method is stable.

Furthermore, since all roots other than the principal root $\lambda = 1$ lie strictly inside the unit circle, the method is

strongly stable.

(b) A linear multistep method is convergent if and only if it is both consistent and stable (zero-stable).

From part (a), the method satisfies the root condition and is therefore stable. We verify consistency in part (c). Since the method is both stable and consistent, it

follows that

the method is convergent.

(c) For a linear multistep method, define

$$\rho(\lambda) = \lambda^3 - \frac{18}{11}\lambda^2 + \frac{9}{11}\lambda - \frac{2}{11},$$

and

$$\sigma(\lambda) = \frac{6}{11}\lambda^3.$$

A linear multistep method is consistent if

$$\rho(1) = 0$$

and

$$\rho'(1) = \sigma(1).$$

First,

$$\rho(1) = 1 - \frac{18}{11} + \frac{9}{11} - \frac{2}{11} = \frac{11 - 18 + 9 - 2}{11} = 0.$$

Next,

$$\rho'(\lambda) = 3\lambda^2 - \frac{36}{11}\lambda + \frac{9}{11}.$$

Therefore,

$$\rho'(1) = 3 - \frac{36}{11} + \frac{9}{11} = \frac{33 - 36 + 9}{11} = \frac{6}{11}.$$

Also,

$$\sigma(1) = \frac{6}{11}.$$

Hence,

$$\rho'(1) = \sigma(1).$$

Both consistency conditions are satisfied, and therefore

the method is consistent.

□

4. Problem 4

Use a built-in ODE solver to numerically integrate the Lorenz equations

$$\begin{aligned} \dot{x} &= \sigma(x - y) \\ \dot{y} &= rx - y - xz \\ \dot{z} &= xy - bz \end{aligned}$$

with the following values for coefficients $\sigma = 10, r = 28, b = 8/3$.

(a) Using the intial condition $(x_0, y_0, z_0) = (5, 5, 5)$, solve the system for $0 \leq t \leq 10$ with a time step $h = 0.001$. Make a 2D plot of x versus z and a 3D plot of x versus y versus z.

(b) Use a new initial condition $(\bar{x_0}, \bar{y_0}, \bar{z_0}) = (5, 5, 5)10^{-5}$, repeat the simulations in part (a) and the plots in part (b).

(c) Plot the L_2-norm pof the difference in the solutions obtained using the different initial conditions in (a) and (b). Comment on your results.

(d) Repeat parts (a) - (c) with different solvers and different step sizes. Comment on your results. You may see differences when using some solvers but not using others. Given the differences observed when using different methods, how can you tell what to expect for the exact solution?

Solution. The standard Lorenz equations are

$$\dot{x} = \sigma(y - x), \tag{1}$$

$$\dot{y} = rx - y - xz, \tag{2}$$

$$\dot{z} = xy - bz, \tag{3}$$

where

$$\sigma = 10, \qquad r = 28, \qquad b = \frac{8}{3}.$$

Therefore, if

$$\mathbf{u}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix},$$

the system can be written as

$$\dot{\mathbf{u}} = \mathbf{f}(t, \mathbf{u}) = \begin{bmatrix} 10(y - x) \\ 28x - y - xz \\ xy - \dfrac{8}{3}z \end{bmatrix}.$$

The equation in the problem statement is written as $\dot{x} = \sigma(x - y)$. In this solution, the standard Lorenz convention $\dot{x} = \sigma(y - x)$ is used.

The notation for the second initial condition is interpreted as a small perturbation of the first initial condition:

$$\mathbf{u}_0 = \begin{bmatrix} 5 \\ 5 \\ 5 \end{bmatrix}, \qquad \bar{\mathbf{u}}_0 = \mathbf{u}_0 + 10^{-5} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

(a) Solution using the initial condition $(5, 5, 5)$**.** The system was first solved with MATLAB's built-in `ode45` solver over

$$0 \leq t \leq 10.$$

The requested output spacing and maximum internal step size were both set to

$$h = 0.001.$$

The MATLAB commands used for parts (a)–(c) are shown below.

```
clear;
close all;
clc;

% Lorenz parameters
sigma = 10;
r = 28;
b = 8/3;

% Lorenz equations
lorenz = @(t,u) [sigma*(u(2)-u(1));
                 r*u(1)-u(2)-u(1)*u(3);
                 u(1)*u(2)-b*u(3)];

% Time interval and initial conditions
h = 0.001;
tspan = 0:h:10;

u0 = [5; 5; 5];
u0bar = u0 + 1e-5*[1; 1; 1];

% Solver options
options = odeset('RelTol',1e-9, ...
                 'AbsTol',1e-12, ...
                 'MaxStep',h);

% Part (a)
[t1,U1] = ode45(lorenz,tspan,u0,options);
```

```
x1 = U1(:,1);
y1 = U1(:,2);
z1 = U1(:,3);

figure;
plot(x1,z1,'b','LineWidth',0.8);
xlabel('x');
ylabel('z');
title('Lorenz system: x versus z');
grid on;
saveas(gcf,'lorenz_xz_a.png');

figure;
plot3(x1,y1,z1,'b','LineWidth',0.7);
xlabel('x');
ylabel('y');
zlabel('z');
title('Lorenz trajectory for u_0=(5,5,5)');
grid on;
axis tight;
view(3);
saveas(gcf,'lorenz_3d_a.png');
```

The resulting x–z phase plot and three-dimensional phase-space trajectory are shown in Figures 2 and 3.

The trajectory displays the characteristic two-lobed structure of the Lorenz attractor. It spirals around one lobe for some time before transitioning to the other lobe.

(b) Solution using the perturbed initial condition. The simulation was repeated using

$$\bar{\mathbf{u}}_0 = \begin{bmatrix} 5 + 10^{-5} \\ 5 + 10^{-5} \\ 5 + 10^{-5} \end{bmatrix}.$$

The following MATLAB code produces the corresponding plots.

```
% Part (b)
[t2,U2] = ode45(lorenz,tspan,u0bar,options);
```

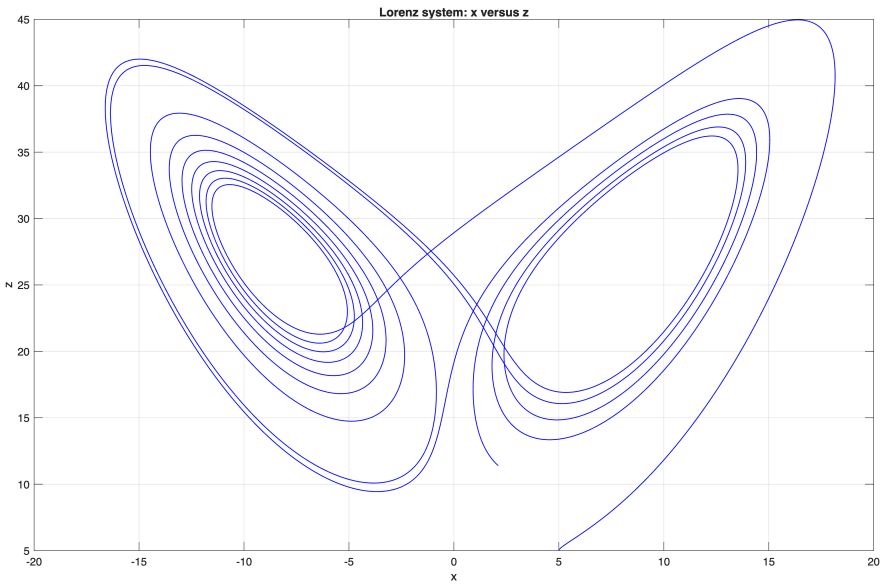

FIGURE 2. The x–z phase plot for $\mathbf{u}_0 = (5, 5, 5)^T$.

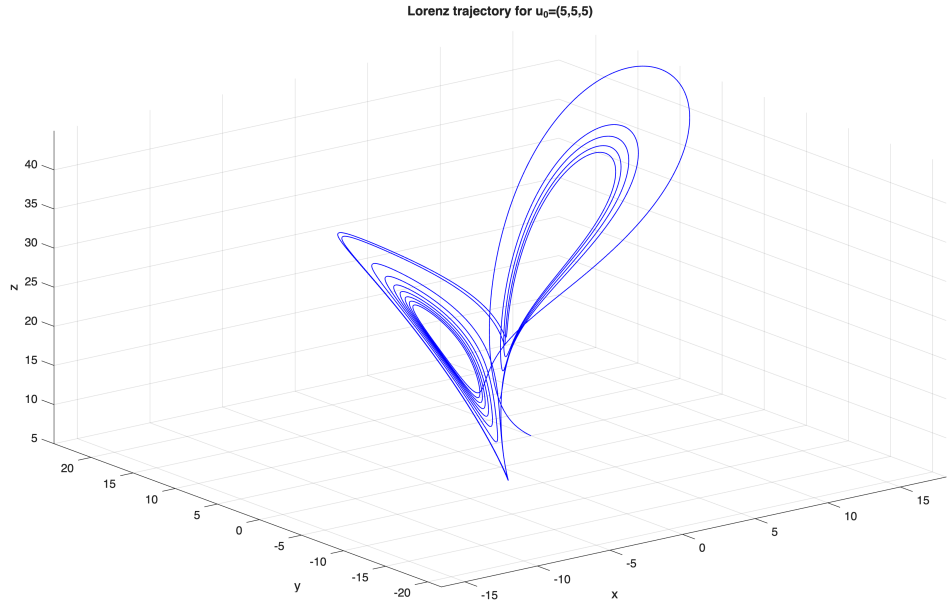

FIGURE 3. Three-dimensional Lorenz trajectory for $\mathbf{u}_0 = (5, 5, 5)^T$.

```
x2 = U2(:,1);
y2 = U2(:,2);
z2 = U2(:,3);

figure;
plot(x2,z2,'r','LineWidth',0.8);
```

```
xlabel('x');
ylabel('z');
title('Perturbed Lorenz system: x versus z');
grid on;
saveas(gcf,'lorenz_xz_b.png');

figure;
plot3(x2,y2,z2,'r','LineWidth',0.7);
xlabel('x');
ylabel('y');
zlabel('z');
title('Lorenz trajectory for perturbed initial condition');
grid on;
axis tight;
view(3);
saveas(gcf,'lorenz_3d_b.png');
```

The corresponding phase plots are shown in Figures 4 and 5.

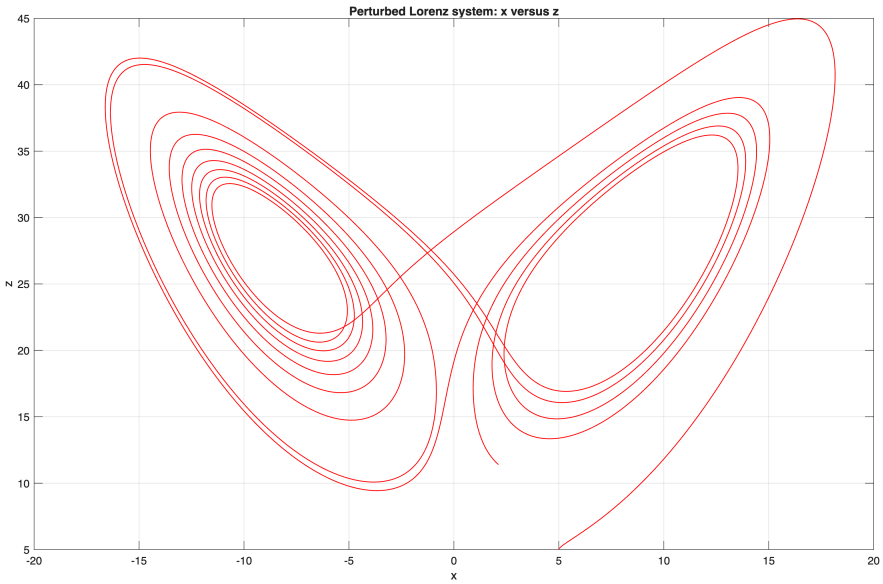

FIGURE 4. The x–z phase plot for the perturbed initial condition.

At first, the trajectory from the perturbed initial condition appears nearly identical to the trajectory from part (a). At later times, however, the two trajectories follow different paths around the two lobes of the attractor.

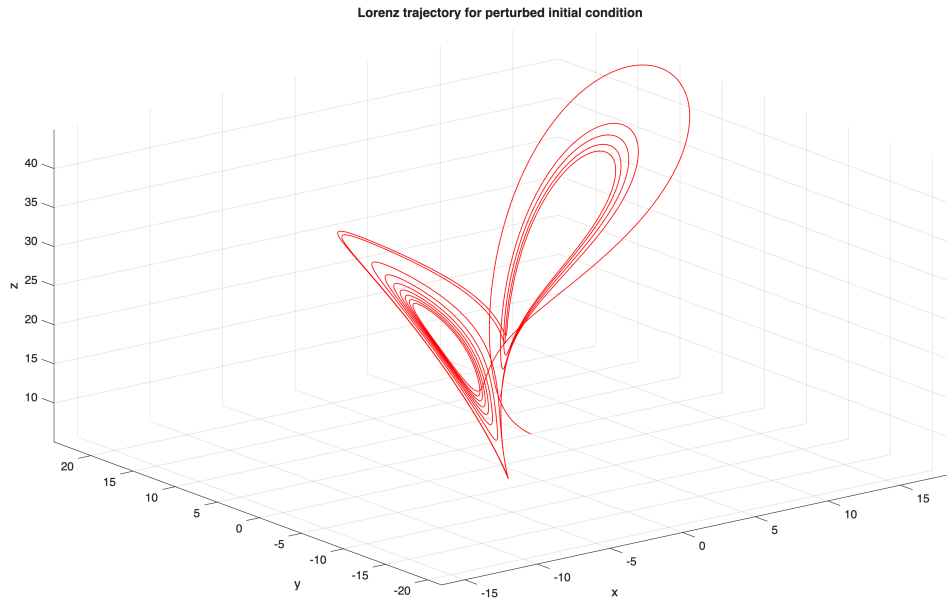

FIGURE 5. Three-dimensional Lorenz trajectory for the perturbed initial condition.

(c) Norm of the difference between the solutions. Let

$$\mathbf{u}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix} \quad \text{and} \quad \bar{\mathbf{u}}(t) = \begin{bmatrix} \bar{x}(t) \\ \bar{y}(t) \\ \bar{z}(t) \end{bmatrix}.$$

The pointwise Euclidean norm of the difference is

$$D(t) = \|\mathbf{u}(t) - \bar{\mathbf{u}}(t)\|_2 \tag{4}$$

$$= \sqrt{\big(x(t) - \bar{x}(t)\big)^2 + \big(y(t) - \bar{y}(t)\big)^2 + \big(z(t) - \bar{z}(t)\big)^2}. \tag{5}$$

The initial separation is

$$D(0) = \left\| 10^{-5} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\|_2 = \sqrt{3} \times 10^{-5}.$$

The norm was calculated and plotted using the following MATLAB commands.

```
% Part (c)
solutionDifference = U1-U2;
L2difference = sqrt(sum(solutionDifference.^2,2));

figure;
semilogy(t1,L2difference,'k','LineWidth',1.2);
xlabel('Time, t');
```

```
ylabel('||u(t)-u-bar(t)||_2');
title('Separation of nearby Lorenz trajectories');
grid on;
saveas(gcf,'lorenz_difference.png');

fprintf('Initial separation = %.6e\n',L2difference(1));
fprintf('Final separation   = %.6e\n',L2difference(end));
fprintf('Maximum separation = %.6e\n',max(L2difference));
```

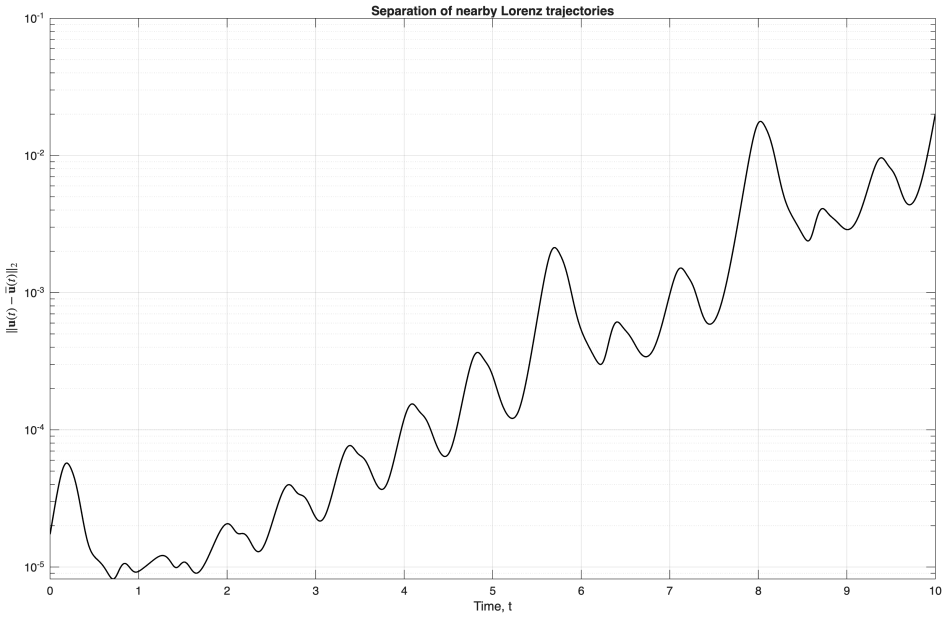

FIGURE 6. The pointwise L_2 norm of the difference between the two Lorenz solutions.

During an initial time interval, the difference grows approximately exponentially:

$$D(t) \approx D(0)e^{\lambda t},$$

where $\lambda > 0$ is related to the largest Lyapunov exponent of the system. Consequently, an approximately straight increasing portion is observed when $D(t)$ is plotted on a logarithmic vertical scale.

Eventually, the difference no longer follows a simple exponential growth law. By this time, the trajectories are at unrelated locations on the attractor, and their separation is comparable to the dimensions of the attractor itself.

This behavior demonstrates sensitive dependence on initial conditions. Although the initial conditions differ by only 10^{-5} in each component, their trajectories eventually become substantially different.

(d) Comparison of solvers and step sizes. The calculations were repeated using the MATLAB solvers

`ode23`, `ode45`, `ode113`, `ode15s`,

and the maximum step sizes

$$h = 0.01, \qquad h = 0.005, \qquad h = 0.001.$$

The following code plots the separation of the nearby trajectories for each solver and step size.

```
solverNames = {'ode23','ode45','ode113','ode15s'};
solvers = {@ode23,@ode45,@ode113,@ode15s};
stepSizes = [0.01,0.005,0.001];

figure;
plotNumber = 1;

for i = 1:length(solvers)
    for j = 1:length(stepSizes)
        currentSolver = solvers{i};
        currentH = stepSizes(j);
        currentTspan = 0:currentH:10;

        currentOptions = odeset( ...
            'RelTol',1e-9, ...
            'AbsTol',1e-12, ...
            'MaxStep',currentH);

        [tA,UA] = currentSolver( ...
            lorenz,currentTspan,u0,currentOptions);

        [tB,UB] = currentSolver( ...
            lorenz,currentTspan,u0bar,currentOptions);

        currentDifference = sqrt(sum((UA-UB).^2,2));

        subplot(length(solvers),length(stepSizes),plotNumber);
```

```
        semilogy(tA, currentDifference, 'LineWidth',0.9);
        grid on;

        title(sprintf('%s, h_{max}=%.3g', ...
              solverNames{i}, currentH));

        if i == length(solvers)
            xlabel('t');
        end

        if j == 1
            ylabel('||Delta u||_2');
        end

        plotNumber = plotNumber+1;
    end
end

sgtitle('Effect of solver and maximum step size');
saveas(gcf,'lorenz_solver_comparison.png');
```

The MATLAB solvers used here are adaptive. Thus, the vector

`tspan = 0:0.001:10`

specifies the times at which MATLAB returns the solution, but it does not force the solver to use an internal step of exactly 0.001. The option

`MaxStep` $= 0.001$

prevents the solver from taking an internal step larger than the specified value.

The numerical experiments show that smaller maximum step sizes and tighter tolerances generally cause the different numerical solutions to agree for a longer time. Higher-order methods such as `ode45` and `ode113` generally give better agreement than lower-order methods for the same error tolerances.

Nevertheless, even very accurate numerical solutions eventually separate. This does not necessarily indicate that a solver is incorrect. The Lorenz system is chaotic, so truncation and roundoff errors are amplified in the same way as a small perturbation of the initial

FIGURE 7. Growth of the difference between nearby trajectories for several MATLAB solvers and maximum step sizes.

condition. Two accurate solvers can therefore produce different pointwise trajectories at sufficiently large times while still producing the same qualitative Lorenz attractor.

There is no simple closed-form exact solution available for comparison. For a fixed finite time interval, confidence in the numerical solution is obtained by a convergence study:

(1) decrease the maximum step size;
(2) tighten the relative and absolute error tolerances;
(3) compare different high-order solvers; and
(4) use higher-precision arithmetic if necessary.

The portion of the trajectory that remains unchanged under these refinements can be regarded as a reliable approximation to the exact solution. Once the refined trajectories begin to separate, the exact long-term pointwise trajectory cannot be predicted reliably using

ordinary floating-point computations. However, the geometry and statistical properties of the Lorenz attractor can still be accurately approximated.

□